TutorConnect Milestone 4 report

Enterprise AI functionalities

Yinqi Chen | CMPE 172 | November 11, 2026

1 AI integration and application design

TutorConnect now has a conversational scheduling assistant, an autonomous reminder agent and policy answers grounded in a small knowledge source. Both agents use Spring AI with the Gemini Developer API. The existing session login, JDBC repositories and transaction services remain responsible for appointment rules.

The conversational agent can query available sessions, list the customer's appointments and propose a booking or cancellation. A separate confirmation request performs the write. The reminder agent runs on a schedule, asks Gemini to generate a study reminder and sends the result through the notification service built in Milestone 3.

Spring AI 2.0.2 is pinned alongside Spring Boot 4.1.1 and Java 17. AiModelConfiguration creates the Google GenAI client with an external API key and vertexAI(false). No project ID or location is configured. The repository default is gemini-3.8-flash, and the local deployment uses gemini-3.1-flash-lite because it completed requests with this account. The embedding model is gemini-embedding-001. Environment variables allow both model names to be changed.

2 Conversational agent and tools

AiChatService uses Spring AI ChatClient and a ToolCallingAdvisor. The model can call a tool, read its result and call another tool before answering. For example, it can query the next seven days, select an available algebra session and propose that session. An unavailable slot produces a conflict result that tells the assistant to query again and offer alternatives.

Tool	Application behavior
checkAvailability	Calls SlotService for a UTC date range of at most 31 days and returns up to 50 sessions, earliest first
listMyAppointments	Calls AppointmentService.customer for the authenticated customer's ID
book	Reads the real slot and creates a booking proposal with a server-generated summary
cancel	Checks ownership and future BOOKED status before creating a cancellation proposal
searchKnowledge	Retrieves policy passages from SimpleVectorStore, including source IDs

Each request creates an AiBookingTools instance bound to the authenticated Account and its server-side conversation. Tool schemas contain appointment and slot parameters but no user ID, role or confirmation flag. The model cannot choose the account whose appointments it accesses.

```
var tools = new AiBookingTools(
    actor, state, slots, appointments, knowledge);
// ChatClient executes the registered tools in its advisor loop.
client.prompt().messages(history).tools(tools).call();
```

The HTTP session keeps up to 20 messages as model context for follow-up questions. A turn allows at most eight model rounds and ten tool executions. The assistant is instructed to clarify ambiguous

dates or subjects and answer in the user's language. The server independently enforces identity and write protection.

Persistent conversation history is the advanced feature for this milestone. AiChatHistory saves each successful exchange in ai_chat_threads and ai_chat_messages through a JDBC transaction. Both tables use InnoDB. The authenticated customer's ID scopes history lists and transcript reads. History survives New chat and another login, while saved transcripts remain read-only. Pending action tokens stay in the HTTP session and are excluded from the saved messages.

The interface shows recent conversations in a desktop sidebar and opens transcripts in a history panel. Reading a transcript preserves the current chat and draft. Drafts use an account-specific key in sessionStorage. The message region scrolls independently, keeping the composer and action confirmation available during long conversations.

A live request exposed a date-format error in checkAvailability. Its date parameters now specify yyyy-MM-dd. If a generated date cannot be parsed, the tool loop returns a correction instruction to Gemini so it can retry within the existing round limit.

3 Confirmation and the transactional write path

A booking tool call does not insert an appointment. It produces a pending action containing the exact slot, service, summary and a random token. The pending action belongs to one customer session, expires after five minutes and is replaced when a new chat turn begins. Only the browser receives its token; the tool result sent back to Gemini contains the summary and confirmation requirement.

The customer reviews the course, tutor and formatted UTC time, then clicks Confirm booking or Confirm cancellation. That sends a CSRF-protected POST to /api/customer/ai/confirm. The endpoint reads the stored action and compares the token, identity and expiry. It accepts no replacement slot or service fields. Saying yes in chat cannot authorize a write.

```
var pending = state.pending;
// Check actor, token and expiry against server-owned state.
state.pending = null; // consume before attempting the write
long id = appointments.book(
    actor.id(), pending.targetId(), pending.serviceId());
```

The confirmed booking calls the existing AppointmentService and its separate BookingTransaction bean. The READ_COMMITTED transaction locks the availability row with SELECT FOR UPDATE, rechecks availability and inserts the appointment. The generated active_slot_id unique key remains the database backstop. A second customer who confirms the same slot receives a 409 conflict after the first transaction wins. The appointment notification is still queued atomically and sent after commit.

Cancellation follows the same confirmation mechanism and calls AppointmentService.cancel. Ownership and future BOOKED status are checked again in the transactional service. Consuming the token prevents duplicate writes after a repeated request. If a conflict occurs or the proposal expires, the customer must request a new proposal. Session state and confirmations are synchronized within the current application instance.

Both customer routes retain CUSTOMER role checks and browser CSRF protection. A provider cannot use the customer assistant, and another login session cannot confirm a copied token. New chat and Discard clear pending actions. New chat starts a fresh model context and preserves saved transcripts.

4 Autonomous reminder agent

AiReminderService has a genuine @Scheduled job with a default delay of 60 seconds. It scans up to 20 BOOKED appointments starting within the next 24 hours and excludes appointments that already have a reminder event. The schedule runs without a chat prompt. Providers can also run it from the dashboard through POST /api/provider/ai/reminders/run; that request scans only the logged-in provider's appointments.

For each candidate, ChatClient asks Gemini to write a short reminder using the customer's name, tutoring subject and UTC start time. It requests a preparation suggestion and rejects blank or oversized output. A model failure is logged and leaves the appointment eligible for another cycle. The code does not substitute fixed text and present it as an AI-generated reminder.

Generation happens before the database transaction. ReminderTransaction then locks and rechecks the appointment, including its BOOKED status and future time, before inserting an APPOINTMENT_REMINDER event. This prevents a cancellation during generation from producing a new reminder.

```
String text = client.prompt()
    .system(reminderInstructions).user(appointmentFacts)
    .call().content();
if (transaction.enqueue(appointmentId, text)) {
    dispatcher.afterCommit(appointmentId, "APPOINTMENT_REMINDER");
}
```

The event ID is appointment:<id>:APPOINTMENT_REMINDER. The outbox primary key and appointment lock prevent duplicate reminder records. NotificationDispatcher uses the existing HTTP gateway with bounded retries and persistent fallback. A failed delivery remains PENDING for recovery. A reminder that was cancelled or has already started before recovery is marked FAILED with NOT_UPCOMING and is suppressed. The receiver's durable event ID deduplication protects repeated HTTP attempts.

The outbox makes delivery durable after generation and enqueue; it does not persist an unfinished LLM request. HTTP delivery is at least once, and the mock stores one receipt per event. It sends no real email or SMS. A cancellation can still race an HTTP request already in flight, so suppression cannot retract a notification already accepted by the receiver.

Scheduling is enabled independently of notification recovery. A two-thread scheduler allows outbox recovery to run while the reminder job waits for a model. The GenAI client has a 20-second request timeout and Spring AI permits one retry. Logs record event or appointment IDs and failure categories without logging keys or full model responses.

5 Knowledge source and grounded answers

The knowledge file knowledge/tutorconnect-policies.md contains six short, hand-written passages about cancellation, available hours, services, booking, reminders and access rules. Its cancellation policy matches the application: customers may cancel their own future BOOKED appointments, while past or cancelled appointments cannot be cancelled again. The file defines no refund policy.

AiKnowledgeService splits the file at its policy headings. Each passage becomes a Spring AI Document whose ID and source metadata identify the policy. On the first search, the embedding model creates real Gemini vectors and SimpleVectorStore stores them in memory. A successful index is reused for the life of the JVM; a failed ingestion is retried on a later request. Restarting the app rebuilds the index when it is next needed.

```
var store = SimpleVectorStore.builder(embeddingModel).build();
store.add(policyDocuments);
var hits = store.similaritySearch(SearchRequest.builder()
    .query(question).topK(3).similarityThreshold(0.35).build());
```

The searchKnowledge tool returns up to three matching passages with source IDs such as policy-cancellation. The assistant receives those passages as reference data and is instructed to cite them, answer only supported policy facts and admit when the source lacks an answer. Live availability and personal appointment questions use their own service tools, because the FAQ cannot establish current database state.

Retrieval grounds the prompt but does not guarantee that every generated sentence is correct. The knowledge file is a small demo policy set. Neither the vector store nor the model is allowed to bypass the server's confirmation and authorization checks.

6 Validation and configuration

The final local validation on November 11, 2026 passed 89 Java tests on Java 17 with H2 and 89 with MySQL 8.4. Both runs had zero failures, errors or skipped tests. Four JavaScript regression tests also passed. The application image built successfully and the deployed app and database passed their health checks.

Test group	Count	Evidence
Booking and application integration suites	57	Role checks, locking, rollback, notifications, cancellation review, provider form recovery and page states
AiWorkflowTests	16	Spring AI tool loop, explicit confirmation, concurrent booking, private saved history, RAG, reminder trigger and invalid-date recovery
AiFailureTests and AiDisabledTests	4	Model errors, bounded loops, reminder failure and disabled AI behavior
AiConfirmationTests	4	Token expiry, identity, role checks and bounded model context
AiReminderDeliveryTests	4	Reminder delivery, deduplication, cancellation checks and provider scope
AiSchedulingTests	2	Scheduling independent of notification recovery and scheduler thread capacity
GeminiAdapterTests	2	Google SDK response handling, signed tool calls and filtering thought summaries

The agent concurrency test uses two independently authenticated clients. Each obtains a proposal for the same slot. Readiness and start latches release both confirmation requests through separate futures. The test asserts one HTTP 200 response, one HTTP 409 response and exactly one active database appointment. The winning write follows the notification outbox path.

The saved-history test starts a new chat, signs out and signs back in, then reads the earlier transcript. Another customer receives 404 for that transcript and a provider receives 403. The test also verifies multi-turn appends and a saved confirmation message. Provider form tests cover duration and overlap errors, including preservation of valid input when another time field cannot be parsed.

Four JavaScript tests cover the immediate user-message preview, replacement after a successful reply, confirmation rendering without an extra history request, and timeout recovery with the original draft retained. Browser checks covered portrait widths of 320 and 390 pixels, a 1366 by 768 desktop viewport, and short landscape viewports. Long conversations retained visible confirmation and send

buttons. History panels preserved drafts and restored keyboard focus. Provider errors remained on the dashboard, and the selected course determined the UTC end time.

Repeatable AI tests supply scripted model responses while exercising the real ChatClient tool loop, HTTP sessions, services and database. GeminiAdapterTests use the actual Google SDK against a local HTTP server. GeminiPublicAnswerModel filters thought summaries, preserves tool-call signatures and combines public answer parts.

Live checks with the locally configured Gemini key returned readable appointment details and a policy answer citing policy-cancellation through vector retrieval. After the date recovery fix, the same Find my next session request completed successfully and reported no algebra sessions in the following seven days. A database query confirmed that result. Live confirmed writes and personalized outbound reminders still require the student's recorded demonstration. Their automated integration tests pass.

AI_ENABLED enables the model configuration and GEMINI_API_KEY supplies the Google AI Studio key. The key is read at runtime from the local environment. The repository and Docker build exclude .env. The browser shows progress and stops waiting after 45 seconds. If a confirmation request times out, the customer should check My appointments before retrying.

The implementation is published in the project's main branch at commit de9d0e8. These results describe local validation. The recorded walkthrough should explain the core code and tests for at least five minutes.

Repository https://github.com/Lushangtu123/CMPE-172-Milestone